Verifica scritta di informatica – Classe 4 AIQ – Compito A
Soluzioni (senza `EXISTS`, `ANY`)

Prof. Fedeli Massimo

Soluzioni – Parte A (Query SQL)

1. **Noleggi del cliente con id_cliente = 7**

```
SELECT n.id_noleggio, m.targa, n.data_inizio, n.
    data_fine,
c.nome, c.cognome
FROM NOLEGGI n
JOIN MOTO m   ON m.id_moto   = n.id_moto
JOIN CLIENTI c ON c.id_cliente = n.id_cliente
WHERE n.id_cliente = 7;
```

2. **Noleggi iniziati a febbraio 2026**

```
SELECT c.cognome, c.nome, m.targa, n.data_inizio, n.
    data_fine
FROM NOLEGGI n
JOIN CLIENTI c ON c.id_cliente = n.id_cliente
JOIN MOTO m   ON m.id_moto   = n.id_moto
WHERE n.data_inizio >= '2026-02-01'
AND n.data_inizio < '2026-03-01';
```

3. **Moto mai noleggiate**

```
SELECT m.id_moto, m.targa, m.marca, m.modello
FROM MOTO m
LEFT JOIN NOLEGGI n ON n.id_moto = m.id_moto
WHERE n.id_noleggio IS NULL;
```

4. **Clienti che hanno noleggiato almeno una Touring**

```
SELECT DISTINCT c.id_cliente, c.cognome, c.nome
FROM CLIENTI c
JOIN NOLEGGI n ON n.id_cliente = c.id_cliente
JOIN MOTO m   ON m.id_moto   = n.id_moto
WHERE m.categoria = 'Touring';
```

5. **Noleggi senza alcun pagamento con esito positivo**

```
SELECT n.id_noleggio, n.id_cliente, n.id_moto,
n.data_inizio, n.data_fine
FROM NOLEGGI n
LEFT JOIN PAGAMENTI p
ON p.id_noleggio = n.id_noleggio
AND p.esito = 'positivo'
WHERE p.id_pagamento IS NULL;
```

6. **Numero totale di noleggi per ogni moto**

```
SELECT m.id_moto, m.targa,
COUNT(n.id_noleggio) AS totale_noleggi
FROM MOTO m
LEFT JOIN NOLEGGI n ON n.id_moto = m.id_moto
GROUP BY m.id_moto, m.targa;
```

7. **Spesa totale per cliente (solo pagamenti con esito positivo)**

```
SELECT c.id_cliente, c.cognome, c.nome,
COALESCE(SUM(p.importo), 0) AS spesa_totale
FROM CLIENTI c
LEFT JOIN NOLEGGI n  ON n.id_cliente = c.id_cliente
LEFT JOIN PAGAMENTI p ON p.id_noleggio = n.
    id_noleggio
AND p.esito = 'positivo'
GROUP BY c.id_cliente, c.cognome, c.nome;
```

8. **Moto con costo totale manutenzione superiore al costo medio**

   Costo intervento: ore_manodopera $\times$ costo_manodopera_oraria $+ \sum$(quantita $\times$ costo_unitario)

```
SELECT t.id_moto, t.targa, t.costo_manutenzione
FROM (
SELECT m.id_moto, m.targa,
COALESCE(SUM(i.ore_manodopera * i.
    costo_manodopera_oraria), 0)
```

```
+ COALESCE(SUM(ir.quantita * r.costo_unitario), 0)
AS costo_manutenzione
FROM MOTO m
LEFT JOIN INTERVENTI i     ON i.id_moto     = m.
    id_moto
LEFT JOIN INTERVENTI_RICAMBI ir ON ir.id_intervento =
     i.id_intervento
LEFT JOIN RICAMBI r        ON r.id_ricambio = ir.
    id_ricambio
GROUP BY m.id_moto, m.targa
) AS t
WHERE t.costo_manutenzione > (
SELECT AVG(t2.costo_manutenzione)
FROM (
SELECT m2.id_moto,
COALESCE(SUM(i2.ore_manodopera * i2.
    costo_manodopera_oraria), 0)
+ COALESCE(SUM(ir2.quantita * r2.costo_unitario), 0)
AS costo_manutenzione
FROM MOTO m2
LEFT JOIN INTERVENTI i2      ON i2.id_moto      = m2.
    id_moto
LEFT JOIN INTERVENTI_RICAMBI ir2 ON ir2.id_intervento
     = i2.id_intervento
LEFT JOIN RICAMBI r2         ON r2.id_ricambio  = ir2
    .id_ricambio
GROUP BY m2.id_moto
) AS t2
);
```

9. **Clienti che hanno noleggiato ogni categoria di moto attiva**

   Idea: contare le categorie distinte noleggiate per cliente e confrontarle con il totale delle categorie attive.

```
SELECT c.id_cliente, c.cognome, c.nome
FROM CLIENTI c
JOIN NOLEGGI n ON n.id_cliente = c.id_cliente
JOIN MOTO m   ON m.id_moto   = n.id_moto
WHERE m.attiva = 1
GROUP BY c.id_cliente, c.cognome, c.nome
HAVING COUNT(DISTINCT m.categoria) = (
SELECT COUNT(DISTINCT m2.categoria)
```

```
FROM MOTO m2
WHERE m2.attiva = 1
);
```

10. **Per ciascun cliente, il noleggio più recente**

    Criterio: data_inizio massima; a parità di data, id_noleggio massimo.

```
SELECT c.id_cliente, c.cognome, c.nome,
n.id_noleggio, n.id_moto,
n.data_inizio, n.data_fine
FROM CLIENTI c
JOIN (
SELECT n1.id_cliente,
MAX(n1.data_inizio) AS max_data_inizio
FROM NOLEGGI n1
GROUP BY n1.id_cliente
) x ON x.id_cliente = c.id_cliente
JOIN NOLEGGI n
ON n.id_cliente = x.id_cliente
AND n.data_inizio = x.max_data_inizio
JOIN (
SELECT n2.id_cliente, n2.data_inizio,
MAX(n2.id_noleggio) AS max_id_noleggio
FROM NOLEGGI n2
GROUP BY n2.id_cliente, n2.data_inizio
) y ON y.id_cliente    = n.id_cliente
AND y.data_inizio   = n.data_inizio
AND y.max_id_noleggio = n.id_noleggio;
```

Soluzioni – Parte B (Algebra Relazionale)

Nota: si usa l'algebra relazionale estesa con l'operatore di raggruppamento γ.

1. **Targhe delle moto attive di categoria "Touring"**

$$\pi_{\text{targa}}\big(\sigma_{\text{attiva}=1 \,\wedge\, \text{categoria}=\text{'Touring'}}(\text{MOTO})\big)$$

2. **Cognome e nome dei clienti registrati nel 2026**

$$\pi_{\text{cognome, nome}}\big(\sigma_{\text{'2026-01-01'} \leq \text{data_reg} < \text{'2027-01-01'}}(\text{CLIENTI})\big)$$

3. **Targhe delle moto che compaiono in almeno un noleggio**

$$\pi_{\text{targa}}\big(\text{MOTO} \bowtie_{\text{MOTO.id_moto=NOLEGGI.id_moto}} \text{NOLEGGI}\big)$$

4. **Coppie (cliente, targa) per i noleggi di febbraio 2026**

$$\pi_{\text{CLIENTI.id_cliente, MOTO.targa}}\big(\sigma_{\text{`2026-02-01'} \leq \text{data_inizio} < \text{`2026-03-01'}}(\text{NOLEGGI}) \bowtie \text{CLIENTI} \bowtie \text{MOTO}\big)$$